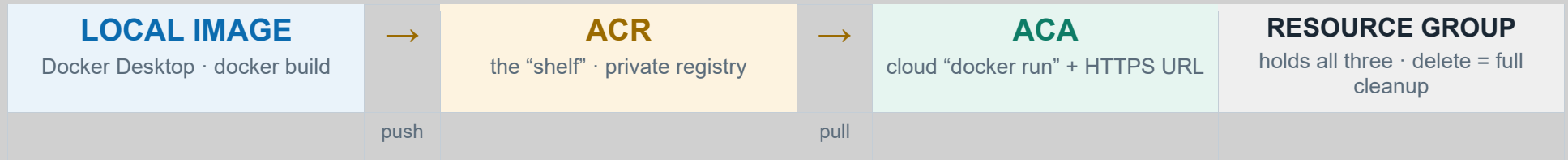


Deploying a Containerised FastAPI App to Azure

Docker image → Azure Container Registry → Azure Container Apps · auth methods, secrets & full command reference

1. The Deployment Flow

Azure cannot run an image that only exists on your laptop. The image must first be pushed to a cloud registry (ACR), which a hosting service (ACA) then pulls and runs.



Docker Desktop mapping: your local image store = ACR · docker run = ACA · your laptop/host = Resource Group. The one real difference is that in Azure the image store and the runner are separate services, which is why an authentication step exists.

2. Why Authentication Is Needed

Your ACR is private by default, so ACA must prove its identity before the registry hands over the image. There are two ways to do this.

Method 1 — Admin credentials (the simple way)

- ACR generates a built-in username (the registry name) and a long random password.
- Read them with: `az acr credential show --name myfastapiacr123`
- Effectively a master key for the whole registry — grants pull AND push.
- Good for learning; Microsoft recommends disabling it for anything real.

Method 2 — Managed Identity (recommended)

- Azure gives ACA its own identity badge; no password exists anywhere.
- User-assigned = created separately, reusable, can hold its role before the app exists (preferred).
- System-assigned = created by the app, deleted with the app.
- Only one role is required: `AcrPull` — read-only, cannot push or delete.

The 4-step identity handshake (order matters)

1. Allow ARM-audience tokens on the ACR.
2. Create or enable the managed identity.

3. Grant that identity the AcrPull role on the registry.
4. Point ACA at ACR using the identity (--registry-identity).

Warning: the identity must exist AND already hold AcrPull before the first pull, otherwise the deployment fails with “access denied”. Allow ~30 seconds for RBAC to propagate.

3. Full Command Comparison

Highlighted rows (6–9) are the only steps where the two methods differ. Rows 14, 18 and 21 differ slightly; everything else is identical.

#	What it does	Method 1 — Admin creds (simple)	Method 2 — Managed Identity (recommended)
1	Sign in to Azure from your terminal	<code>az login</code>	<i>identical to Method 1</i>
2	Install / refresh the Container Apps CLI extension	<code>az extension add --name containerapp --upgrade</code>	<i>identical to Method 1</i>
3	Register the resource providers ACA needs	<code>az provider register --namespace Microsoft.App</code> <code>az provider register --namespace Microsoft.OperationalInsights</code>	<i>identical to Method 1</i>
4	Create the box that holds everything (one delete = full cleanup)	<code>az group create -n my-fastapi-rg -l centralindia</code>	<i>identical to Method 1</i>
5	Create the shelf for the image (Basic = cheapest tier)	<code>az acr create -g my-fastapi-rg -n myfastapiacr123 --sku Basic</code>	<i>identical to Method 1</i>
6	AUTH PREP — enable the credential mechanism this method needs	<code>az acr update -n myfastapiacr123 \</code> <code>--admin-enabled true</code> # turns on the shared admin user	<code>az acr config authentication-as-arm update \</code> <code>-r myfastapiacr123 --status enabled</code> # allows ARM tokens, required for MI pull
7	AUTH IDENTITY — read the credential / create the badge	<code>ACR_USER=\$(az acr credential show -n myfastapiacr123 \</code> <code>--query username -o tsv)</code> <code>ACR_PASS=\$(az acr credential show -n myfastapiacr123 \</code> <code>--query "passwords[0].value" -o tsv)</code>	<code>az identity create -g my-fastapi-rg \</code> <code>-n id-fastapi-acrpull</code> # reusable user-assigned badge
8	AUTH IDs — capture values needed for the trust link	<i>(none – user + pass already held in variables)</i>	<code>IDENTITY_ID=\$(az identity show -g my-fastapi-rg \</code> <code>-n id-fastapi-acrpull --query id -o tsv)</code> <code>PID=\$(az identity show -g my-fastapi-rg \</code>

#	What it does	Method 1 — Admin creds (simple)	Method 2 — Managed Identity (recommended)
9	AUTH TRUST — grant read-only pull permission	<i>(not needed – the password IS the permission)</i>	<pre>-n id-fastapi-acrpull --query principalId -o tsv) ACR_ID=\$(az acr show -n myfastapiacr123 --query id -o tsv) az role assignment create \ --assignee-object-id \$PID \ --assignee-principal-type ServicePrincipal \ --scope \$ACR_ID --role "AcrPull" sleep 30 # wait for RBAC propagation</pre>
10	Log Docker into the registry so you can push	<code>az acr login -n myfastapiacr123</code>	<i>identical to Method 1</i>
11	Tag the local image with the ACR login server	<code>docker tag my-fastapi-app:latest \ myfastapiacr123.azurecr.io/app:v1</code>	<i>identical to Method 1</i>
12	Push the image up to the shelf	<code>docker push myfastapiacr123.azurecr.io/app:v1</code>	<i>identical to Method 1</i>
13	Create the ACA environment (secure network boundary)	<code>az containerapp env create -n my-fastapi-env \ -g my-fastapi-rg -l centralindia</code>	<i>identical to Method 1</i>
14	DEPLOY — the cloud docker run; note the auth flags differ	<pre>az containerapp create -n my-fastapi-app \ -g my-fastapi-rg --environment my-fastapi-env \ --image myfastapiacr123.azurecr.io/app:v1 \ --registry-server myfastapiacr123.azurecr.io \ --registry-username \$ACR_USER \ --registry-password \$ACR_PASS \ --target-port 80 --ingress external \ --min-replicas 0 --max-replicas 2</pre>	<pre>az containerapp create -n my-fastapi-app \ -g my-fastapi-rg --environment my-fastapi-env \ --image myfastapiacr123.azurecr.io/app:v1 \ --registry-server myfastapiacr123.azurecr.io \ --registry-identity \$IDENTITY_ID \ --user-assigned \$IDENTITY_ID \ --target-port 80 --ingress external \ --min-replicas 0 --max-replicas 2</pre>
15	SECRETS — store the .env API key encrypted (name: lowercase + hyphen only)	<code>az containerapp secret set -n my-fastapi-app \ -g my-fastapi-rg \ --secrets openai-api-key="sk-your-real-key"</code>	same — or via Key Vault: <pre>--secrets "openai-api-key=keyvaultref:\ https://my-kv.vault.azure.net/secrets/openai-api-key,\ identityref:\$IDENTITY_ID"</pre>
16	ENV VARS — map secret to the name your code calls in <code>os.getenv()</code>	<pre>az containerapp update -n my-fastapi-app -g my-fastapi-rg \ --set-env-vars "OPENAI_API_KEY=secretref:openai-api-key" \ "MODEL_NAME=gpt-4o-mini"</pre>	<i>identical to Method 1</i>
17	Fetch the live public HTTPS URL	<code>az containerapp show -n my-fastapi-app -g my-fastapi-rg \</code>	<i>identical to Method 1</i>

#	What it does	Method 1 — Admin creds (simple)	Method 2 — Managed Identity (recommended)
		<pre>--query "properties.configuration.ingress.fqdn" -o tsv</pre>	
18	Change registry auth later on an existing app	<pre>az containerapp registry set -n my-fastapi-app \ -g my-fastapi-rg \ --server myfastapiacr123.azurecr.io \ --username \$ACR_USER --password \$ACR_PASS</pre>	<pre>az containerapp registry set -n my-fastapi-app \ -g my-fastapi-rg \ --server myfastapiacr123.azurecr.io \ --identity \$IDENTITY_ID</pre>
19	Apply a changed secret value (secrets do NOT auto-restart)	<pre>az containerapp revision restart -n my-fastapi-app \ -g my-fastapi-rg --revision <revision-name></pre>	<i>identical to Method 1</i>
20	Delete everything in one shot (cost cleanup)	<pre>az group delete -n my-fastapi-rg --yes --no-wait</pre>	<i>identical to Method 1</i>
21	Security hygiene — turn the master key back off	<pre>az acr update -n myfastapiacr123 --admin-enabled false</pre>	<i>(nothing — no key ever existed)</i>

4. One-Shot Deploy (steps 14 + 15 + 16 combined)

```
az containerapp create \
-n my-fastapi-app -g my-fastapi-rg --environment my-fastapi-env \
--image myfastapiacr123.azurecr.io/app:v1 \
--registry-server myfastapiacr123.azurecr.io \
--registry-identity $IDENTITY_ID --user-assigned $IDENTITY_ID \
--target-port 80 --ingress external --min-replicas 0 --max-replicas 2 \
--secrets "openai-api-key=sk-your-real-key" \
--env-vars "OPENAI_API_KEY=secretref:openai-api-key" "MODEL_NAME=gpt-4o-mini"
```

5. What Actually Differs

Aspect	Admin creds	Managed Identity
Extra auth commands	2	4
Secret stored anywhere?	Yes — username + password	No — none exists

Aspect	Admin creds	Managed Identity
Registry access scope	Pull AND push (full access)	AcrPull — read-only
Key deploy flag	--registry-username / --registry-password	--registry-identity / --user-assigned
Typical failure	Password rotated → pull breaks	Deployed before RBAC propagated → access denied
Best for	Learning, quick demos	Real projects, shared work, resume pieces

6. Handling the .env API Key

Do not ship the .env file. Keep it for local development and define the same values as Container Apps secrets in the cloud. Your Python code does not change, because `os.getenv()` reads from the environment either way.

Why .env fails in the container

- .env is usually in .dockerignore / .gitignore, so it never reaches the image → `os.getenv()` returns `None`.
- If you do COPY it in, the API key is baked into the image layers and readable by anyone who can pull it.

Code stays the same

```
import os
from dotenv import load_dotenv
load_dotenv()          # local: fills env from .env | Azure: no-op, file absent
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

Env var vs secret — pick the right one

Type	Visible in portal / CLI?	Use it for
Env var (plain)	Yes — shown in config	MODEL_NAME, LOG_LEVEL, MAX_ROWS, PORT
Secret + secretref:	No — encrypted & hidden	API keys, connection strings, passwords

Naming rule: secret names must be lowercase alphanumeric and hyphens only. `openai-api-key` is valid; `OPENAI_API_KEY` is rejected. The environment variable name, however, must match your code exactly (case-sensitive).

Doing it manually in the Azure Portal

Part A — add the secret

1. Open your container app in portal.azure.com.
2. Left nav → Settings → Secrets.
3. Select Add.
4. Key = openai-api-key · Type = Container Apps Secret · Value = your real key.
5. Select Add. The value is stored encrypted and hidden afterwards.

Part B — expose it as an environment variable

1. Left nav → Revisions → Containers.
2. Click Edit and deploy at the top.
3. In Container image, click your container to open Edit a container.
4. Scroll to Environment variables → Add.
5. Name = OPENAI_API_KEY · Source = Reference a secret · Value = openai-api-key.
6. Add plain vars the same way with Source = Manual entry.
7. Save, then Create to deploy the new revision.

7. Gotchas & Cost Notes

- Updating a secret does NOT restart the app — secrets are app-scoped, not revision-scoped. Restart the revision or deploy a new one.
- Add .env and .env.* to .dockerignore so the file never enters the image.
- Container storage is ephemeral — stream the generated Excel back in the response or write it to Azure Blob Storage; do not rely on local disk.
- Large CSVs plus AI compute can hit request timeouts — consider FastAPI background tasks or a job-id pattern.
- --min-replicas 0 enables scale-to-zero, so you pay close to nothing while the API is idle.
- ACR Basic is a small fixed monthly cost; set a budget alert in Cost Management.
- Clean up everything with one command: `az group delete -n my-fastapi-rg --yes --no-wait`

Rule of thumb: *learn with admin credentials, ship with managed identity. Keep .env local, use secrets in the cloud.*